

Honors Lab: Microprocessor on the Board

Prerequisites: Before beginning this lab, you must:

- Understand how to use the tool flow (See the installation guide and Lab 0)
- Have completed Lab 1: Half Adder, Full Adder, 4-bit Incrementer and Adder.
- Have completed Lab 2: Multiplexers, Decoders, and the Arithmetic Logic Unit.
- Have completed Lab 3: Registers, Counters, and the “Brainless CPU”.
- Have completed Lab 4: “The Complete Microprocess”.

Equipment: Personal computer with the required software installed.

Files to copy from Lab 4 (do NOT copy from Lab 1 or Lab 2 or Lab 3!):

addr_gen.dig
alu.dig
and_add.dig
brainless.dig
controller.dig
four_bit_adder.dig
four_bit_mux.dig
four_bit_reg.dig
full_adder.dig
half_adder.dig
incrementer.dig
microprocessor.dig
not_neg.dig
program_ctr.dig
program_ram.dig
two_bit_bus_mux.dig
two_bit_mux.dig
two_bit_reg.dig
rom_vals.hex
ram_vals.hex
ram_vals.txt

File download:

DE10_Lite.config

Objectives: When you have completed this lab, you will be able to:

- Create a lookup table.
- Add a JUMP instruction to your microprocessor.
- Add a BRANCH IF ZERO instruction to your microprocessor.
- Simulate a microprocessor on a board.
- Write a program with a loop.

Introduction

In this lab we will add two instructions to the microprocessor design. We will then add a module which translates hex values to seven bits to drive a seven-segment display. Finally, we will move the design onto an FPGA board and run the design on the board. Because this is an honors project, not all details will be provided.

Warning: Use the signal and circuit names provided! Verilog does not allow names to start with a number or names that have dashes!

NOTE: You will need to run your design on a board in the lab, which is located in GWC 273. You will need to demo it in order to get credit.

There will be at least one review session offered where you can ask questions about this project.

Create a folder named Honors. Into that folder, copy the files listed above from Lab 4. Be sure to only copy the required files. While it is tempting to do the Honors lab in the same directory where you did a prior lab, it is advisable to start in a fresh directory in case something goes wrong. That way, you still have the pristine prior lab results to copy again. In addition, this means that the Honors folder will only have the files necessary for the Honors lab.

Once you've copied the files from Lab 4, you're ready to start!

NOTE: You are required to design the circuits as presented in this document. Even though Digital supplies many of the functions we will design, you are not to use them.

Figure 1 shows the how the top-level will look when you are done.

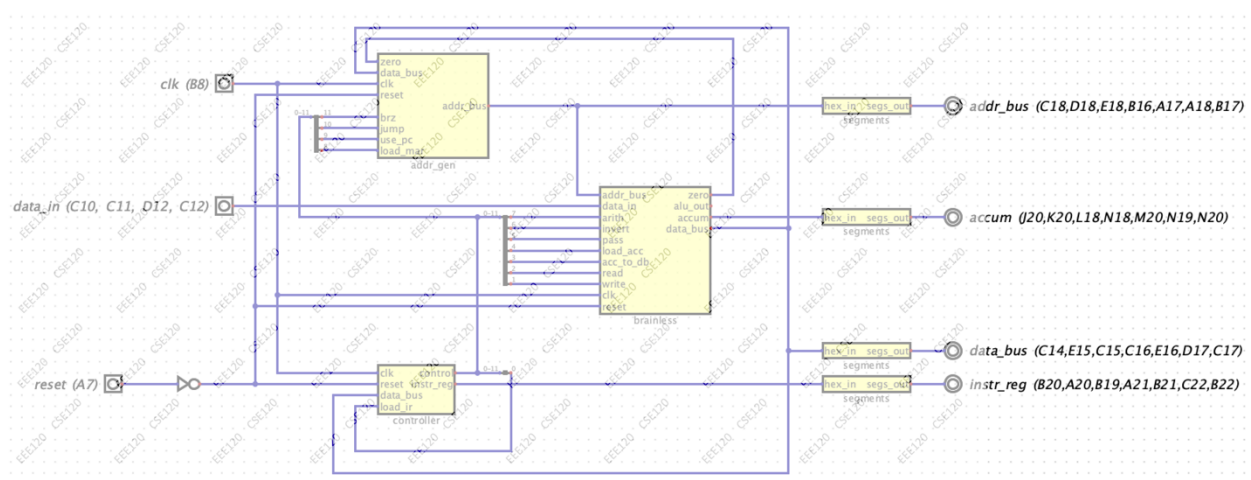


Figure 1. An example of a completed design.

Task H-1: Add the JUMP Instruction

One of the most commonly used instructions in programs is the JUMP instruction. You need to add the JUMP instruction to the microprocessor. Give the JUMP instruction opcode 0x8. The value immediately following the JUMP instruction is one less than the address in the RAM where to find the next instruction. That is, the incrementer in the program counter module should come **after** the mux you need to add!

To implement the JUMP instruction, you will need to add another bit to the microcode ROM that causes the Data Bus to be loaded into the Program Counter. Use bit 10, and call this signal **jump** on the input to the address generation module. But call the signal **load_pc** inside the program counter. (You'll see why it has a different name in the next task.) Figure 2 shows the pinout of the program_ctr module.

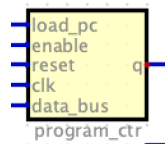


Figure 2. The updated program_ctr interface.

Here is example code showing how the value provided to the JUMP instruction is one less than the target address:

```
0x8  # JUMP instruction
0x3  # causes a JUMP to address 4
```

NOTE: When you edit rom_vals.hex, when editing the controller, be sure to open the ROM and click Edit then click File, and select the rom_vals.hex file in the Honors directory. (If you don't, Digital will still be referencing the rom_vals.hex file in your Lab 4 directory!)

Test the new instruction by creating a program in ram_vals.hex for the Program RAM and simulating it in Digital.

Why is the specified address one less than the address to which we want to jump? As you'll learn in CSE230/EEE230, it is common in assembly language for the jump address to be further altered by the hardware before it is placed in the program counter.

Task H-2: Add the BRANCH ON ZERO Instruction

Now that the JUMP instruction is working, let's add another flavor of jump, which will take the jump only if the accumulator contains 0. This is what is known as a conditional branch. Give the BRANCH ON ZERO instruction, or BRZ for short, the opcode 0x9. As with the JUMP command, the address specified is to be one less than the desired target address.

To implement the BRZ, you'll need to add an output named **zero** to the brainless module that is a 1 if the value of accum is 0. Then route that signal to the program counter. Then, use bit 11 in

the microcode ROM as the **brz** signal to indicate a branch on zero should be taken. Then, combine the **brz**, **zero**, and **jump** inputs to drive the **load_pc** pin on the program counter. (No further change to the program counter is needed.)

Make sure you reload the ROM code after you add the new microcode to `rom_vals.hex`.

Example code:

```
0x0  # Load ACC
0x1  # a nonzero value
0x9  # BRZ instruction
0x9  # not taken since accum is not 0
0x0  # LOAD ACC
0x0  # zero!
0x9  # BRZ instruction
0x9  # branch to address 0xA
0x0  # Load ACC
0x5  # this should be skipped!
0x2  # STOP
```

Test the new instruction by creating the above program in the Program RAM and simulating it in Digital.

Prepare two `ram_vals.hex` files to test the new functionality in Digital. One of these tests may be the example above. The other must be a loop where a value on the `data_in` input is loaded into the accumulator. (Do you need another instruction to get that value into the accumulator? If so, add it! This could be opcode 0xA.) Create a loop where 1 is subtracted from the input and then you use the BRZ to break out of the loop when the accumulator is 0 and, if it's not 0, use JUMP to return to the top of the loop.

When you are satisfied with your two tests, use them to create `ram_vals.txt` files similar to how you did that for Lab 4. We'll need those tests so we can run them on the board. Call the two `ram_vals.txt` files `ram_vals_1.txt` and `ram_vals_2.txt` so you can have them both handy. We'll copy them over `ram_vals.txt` so we can use each in turn.

Task H-3: Build the Seven-Segment Display Translator

When we run the design on the board, we will want to view various values on the seven-segment displays. Unfortunately, the board requires that the segments be controlled individually. Therefore, a translator from hex to a 7-bit bus is required. We'll use a lookup table to implement this logic.

Each segment of the seven-segment displays are off if they are not driven. To turn on a segment, we need to drive its input to 0. The segments are numbered as shown in Fig. 3. The 7-bit field is such that bit 0 is on the right and bit 6 is on the left. For example, 0000000 will cause

the number 8 to be displayed, while 1111000 will display the number 7. For convenience, hex is used to specify the values in the lookup table, so the above will be 0x00 and 0x78. (In the table, hex values of more than one digit have the prefix 0x to remind you they are hex digits. But hex values of only one digit do not display the prefix. This is a bit annoying as it is inconsistent, but it is what it is.) However, when entering values, be in the habit of using the prefix so that single digit hex values greater than 9 are handled correctly.

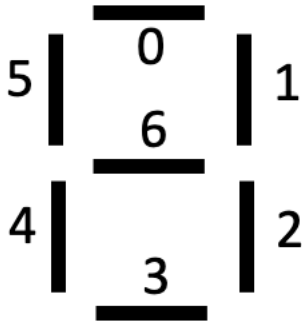


Figure 3. The bit assignments for the seven-segment displays on the board.

Select File->New and then File->Save As and name the file segments. Make sure it saves in the Honors directory! Now select Components->Logic->Lookup Table and place the component on your schematic. Now open the component. Set it to have 7 data bits and 4 inputs. Set the label to be segmenter. Then click Edit. Enter the values shown in Fig. 4 and click OK.

Address	Value
0x0	0x40
0x1	0x79
0x2	0x24
0x3	0x30
0x4	0x19
0x5	0x12
0x6	2
0x7	0x78
0x8	0
0x9	0x10
0xA	8
0xB	3
0xC	0x46
0xD	0x21
0xE	6
0xF	E

Figure 4. The values to enter into the lookup table.

Finally, create the rest of the segments module as shown in Fig. 5. Note that segs_out has 7 bits! Set the width of the segments module to be 8. (Do this in Edit->Circuit specific settings.)

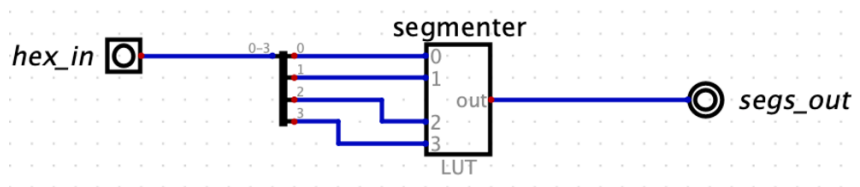


Figure 5. The segments module.

Add 4 copies of the segments module to the microprocessor module and add an output port for each of the segments modules. We'll use them to output the instruction register, the program counter, the data bus, and the accumulator

Task H-4: Bring the Instruction Register to the Top

When running on the board, it will be handy to see the contents of the Instruction Register. In this step, add an output to the controller to bring the instruction register to the top level. Do this by replacing the **instr_reg** tunnel with an output. Call this signal **instr_reg**. Connect this signal to one of the segments modules.

Task H-5: Invert the Reset

We will be using the push buttons on the board for the clock and reset inputs. However, the push buttons on the board are designed so that they send a 1 to the circuit when not pressed, and a 0 when they are pressed. Therefore, we need to invert the reset signal so that the circuit only resets when the button is pressed.

Add an inverter to the reset input of your circuit. Note that this means you'll need to set the reset input to 1 when trying to simulate in Digital.

Task H-6: Connect the Remaining Segments Instances and Assign Pins

In order to run on the board, we need to assign pins to the circuit inputs and outputs so they are properly connected. The reference for the various pins is on the assignment page and is called DE10-Lite_User_Manual_pins.pdf. You can examine it if you wish, but all the information you need to complete this project is found below.

Open the **clk** input and click on Advanced. You'll see a field called Pin number. In that field, enter B8. This assigns the **clk** input to pin B8. Note that this is case sensitive, so the B must be uppercase.

Now open the **reset** input and assign it to pin A7.

Connect the **data_in** input to pins C10, C11, D12, C12. Simply put those pin names, in this EXACT order, into the field separated by commas. This connects the rightmost four switches on

the board to the **data_in** bus. The most significant bit will be on the left, the least significant on the right.

Next, connect the **accum** signal to a segments instance, and connect the output of that segments instance to a new output called **accum**. (This replaces the old **accum** output.) Recall that the segment outputs are 7 bits wide. Assign this output to pins J20, K20, L18, N18, M20, N19, N20. This will place the accumulator output in the LEFT seven-segment display.

Next, connect the **data_bus** to another of the segments instances and name its output **data_bus**. Connect this one to pins C14, E15, C15, C16, E16, D17, C17. This will be the seven-segment display on the right.

Now, connect the **addr_bus** to another segments module, naming its output **addr_bus**. Connect this one to pins C18, D18, E18, B16, A17, A18, B17. This will map the **addr_bus** to the seven-segment display immediately to the left of the one showing the **data_bus**.

Finally, connect the **instr_reg** to the last segments module, naming its output **instr_reg**. Connect this one to pins B20, A20, B19, A21, B21, C22, B22. This will map the **instr_reg** to the seven-segment display immediately to the left of the one showing the **addr_bus**.

You may feel free to use the remaining 2 seven-segment displays to show other values. The pins for the display immediately to the left of the one showing the **instr_reg** are F21, E22, E21, C19, C20, D19, E17. The pins for the display immediately to the right of the one showing the accumulator are F18, E20, E19, J18, H19, F19, F20.

You may also wish to use some of the other switches which are available. The four switches on the right are used for **data_in**, but the others may be used if you wish. The pin mapping for those, from left to right are:

SW9 is pin F15
SW8 is pin B14
SW7 is pin A14
SW6 is pin A13
SW5 is pin B12
SW4 is pin A12

Task H-7: Configure Digital to Create a tcl File

The DE10_Lite.config file must be placed where Digital can find it. Find where Digital is installed on your computer. The DE10_Lite.config file must be placed in the Digital/examples/hdl folder. Move it there.

In Digital, open Edit->Settings and click on Advanced. Click on the three dots to the right of Toolchain Configuration, navigate to the DE10_Lite.config file, and select it then click OK. You

will be warned that you must exit Digital for the change to be effective. Make sure your design is saved and then go ahead and quit and then open your design again in Digital.

There should now be a new menu selection along the top bar, DE10-Lite. Click on that and select the only option: Create build script only. This will save the microprocessor.v file and create a tcl script and an sdc file.

We have the same additional step as in the prior labs since the RAM file is not exported with the Verilog. Again, you'll need to edit microprocessor.v in order to use your RAM contents. Edit the file and place these lines in the DIG_RAMDualPort module. (See the Lab 3 manual for more details.)

```
initial
begin
    $readmemh("ram_vals.txt",memory);
end
```

Task H-8: Send the Design to the Board

Make a new folder and copy these files into the folder:

```
microprocessor.sdc
microprocessor.tcl
microprocessor.v
ram_vals_1.txt
ram_vals_2.txt
```

These are the only files required to finish the lab. Go to the lab, GWC 273.

Log into one of the computers and open a Windows Powershell.

If you are able, put the file with your files on a flash drive. When you plug the flash drive into the lab computer, it should come up as being on the D: drive. You can execute:

```
cd D:\
```

and that will put you on the flash drive. You can then cd to the folder with the files.

If you don't have a flash drive, upload the folder to your google drive so you can download it to the lab computer and place it on the Desktop. In the Windows Powershell, execute:

```
cd Desktop
```

and then cd into the folder with the your files are.

Copy ram_vals_1.txt to ram_vals.txt with the command:

```
cp ram_vals_1.txt ram_vals.txt
```

Now, execute two commands. The first one will take a while. The second should be pretty fast. Once the second one completes, the board attached to the computer should stop flashing and the seven-segment displays will all be 0. The commands are:

```
quartus_sh -t .\microprocessor.tcl  
quartus_pgm -c1 -mjtag -o"p;microprocessor.sof"
```

Task H-9: Run on the Board

Place the board so that the seven-segment displays are on the side closest to you. There should be a number of switches to the right of the seven-segment displays. Along the right side, just above the rightmost switch are two push buttons.

The values on the seven-segment displays are as described above:

accum	unused	unused	instr_reg	addr_reg	data_bus
-------	--------	--------	-----------	----------	----------

As for the switches, the four switches on the right form the **data_in** value. When the device is in reset, as it is when it first starts up, the **data_in** value is on the **data_bus**. Check this out by changing the switches and notice how the right-most seven-segment display changes accordingly. When the switch is in the position toward you, it is a 0. When it is away from you, it's a 1. The value on the **data_in** bus will also appear on the **data_bus** when the STOP command is executing.

The upper push button, which is further from you, controls the clock. Pressing the button causes it to change the clock to 0, releasing it changes the clock to 1.

The lower push button, which is closer to you, controls the reset input. Pressing causes the **reset** signal to go to 0 while releasing it changes the **reset** to 1. Recall that an inverter was added to the **reset** input so that a reset will only happen when you push this button.

Execute a program by pressing the clock signal and, if necessary, flipping the **data_in** switches.

Demo the test for the TA or UGTA. Make sure they take down your name. Then, copy ram_vals_2.txt to ram_vals.txt, with this command:

```
cp ram_vals_2.txt ram_vals.txt
```

and run the two quartus commands again. The second test is now loaded onto your board. Demo this test for the TA or UGTA. Again, make sure they take down your name.

Task H-10: Create a video and submit your report.

Take screenshots of the modules in Digital that are required for the template and paste them in. Make sure you answer all the questions.

Create a video showing your schematics in Digital and explain how your design works. Be sure to show yourself in the video and show your screen. Make sure you show and explain the changes you made from Lab 4 to get the honors project working. Place a link to the video in your template. Be sure to include any required password or to set permissions so that everybody can see your video.

Make sure all of your files are in the Honors directory. Create a zip file of the Honors directory. Turn in the zip file and your completed template. (Double check that you turned in the completed template and NOT the blank one you downloaded. Unfortunately, turning in the wrong template file is a common mistake.)

Turn in your template as a PDF. If you turn in any other format, it might not be graded!

Congratulations! You've completed the Honors project!

